

Herald

Jede Warnmeldung erreicht ihr Ziel - ohne Befehlssyntax.

Zusammenfassung

Herald verarbeitet Systemereignisse und verteilt sie zuverlässig an mehrere Benachrichtigungskanäle, sodass jede Warnmeldung den richtigen Empfänger erreicht. Nutzer:innen kommunizieren in natürlicher Sprache; Herald leitet die Absicht durch ein dreistufiges Verfahren ab (Befehls-Schnellpfad → LLM-Intent-Inferenz → Klärungsrückfall).

Kurzbeschreibung

Ein System zur Ereignisaufnahme und Mehrkanal-Benachrichtigungsverteilung. Herald leitet Systemereignisse zuverlässig an die richtigen Ziele in verschiedenen Messenger-Kanälen weiter und ermöglicht es Nutzer:innen, in natürlicher Sprache zu kommunizieren – die Absicht wird dabei über einen Befehls-Schnellpfad, LLM-Inferenz und einen Klärungsrückfall aufgelöst.

Ausführliche Beschreibung

Herald ist das Benachrichtigungsrückgrat, das sicherstellt, dass ein Systemereignis tatsächlich dort ankommt, wo ein Mensch darauf reagieren kann – die unscheinbare, aber missionskritische Schicht, in der die meisten selbstgebauten Alerting-Systeme stillschweigend versagen. Es nimmt Ereignisse auf und verteilt sie zuverlässig über mehrere Benachrichtigungskanäle, wodurch typische Fehlerquellen ausgeschlossen werden: Warnmeldungen, die verloren gehen, an tote Kanäle weitergeleitet oder im Rauschen untergehen, bis es zu spät ist. Doch zuverlässige Zustellung ist nur die halbe Miete; die andere Hälfte betrifft die Interaktion, wenn ein Mensch reagieren möchte. Hier lehnt Herald den üblichen Kompromiss ab, bei dem Nutzer:innen eine starre Befehlssyntax auswendig lernen müssen, um mit einem Alerting-Bot zu kommunizieren. Stattdessen schreiben sie einfach

in natürlicher Sprache, und Herald ermittelt ihre Absicht durch ein bewusst dreistufiges Verfahren: einen Schnelleitpfad, der explizite Befehle sofort erkennt, gefolgt von LLM-basierter Intent-Inferenz (über Claude Code) für frei formulierte Nachrichten, und schließlich einen *Klärungsrückfall*, der bei echter Mehrdeutigkeit mit einer Rückfrage antwortet, markiert und nachfragt. Diese „Erkennen → Ableiten → Klären“-Leiter ist die gesamte Designphilosophie im Kleinen – der Standardfall bleibt sofort und deterministisch, der flexible Fall wird von einem Modell bearbeitet, und der unklare Fall wird nie durch eine blinde Vermutung gelöst, die die falsche Aktion auslöst.

Herald modelliert zudem Teilnahme und Zuordnung: Eine Umgebungsvariable für den Operator-Benutzernamen (HERALD_<KANAL>_OPERATOR_USERNAME) sowie ein Teilnehmer- und Zuordnungsvertrag steuern die Felder `created_by/assigned_to` und die Benachrichtigungs-@-Markierungen, sodass klar ist, wer was getan hat und wer benachrichtigt wird. In Sachen Governance übernimmt Herald das Helix Constitution als ko-lokalisiertes Submodul und folgt dessen Regeln. Zudem ist es ein früher Produktionsnutzer von Docs Chain – sein vollständiger 66-Dokumente-Markdown→HTML/PDF/DOCX-Korpus wird über Docs Chain-exec:-Transformationen verarbeitet und auf Fehlerfreiheit geprüft. Herald besteht hauptsächlich aus Shell/Go-Tooling mit gestaffelten Spezifikationen (V1→V2→V3→V4-Ablösung) und kanalbezogenen Operator-Setupanleitungen für Messenger und LLM/Agent-Dispatcher.

Warum wir es entwickelt haben

Warnmeldungen scheitern leise – sie werden an den falschen Kanal gesendet, gehen verloren oder erfordern eine starre Befehlssyntax, die sich niemand merken kann. Herald wurde entwickelt, um zuverlässige Verteilung zu garantieren und Nutzer:innen die Kommunikation in natürlicher Sprache zu ermöglichen, sodass Benachrichtigungen sowohl verlässlich als auch mühelos umsetzbar sind.

Warum es ein Game-Changer ist

Es vereint zwei Dinge, die normalerweise als separate Produkte erworben werden – zuverlässiges Multi-Channel-Event-Routing und eine natürlchsprachliche Schnittstelle – zu einem System, in dem Bediener einfach sprechen und die Software versteht, was sie meinen. Der entscheidende Sicherheitsmechanismus, der es für den Produktionseinsatz vertrauenswürdig macht, ist die

Klarstellungsfunktion: Ein Alarmierungssystem, das lieber nachfragt als falsch zu feuern, ist eines, das man tatsächlich an echte Zustände anbinden kann.

Was innovativ ist

- Dreistufige Intent-Disziplin: Befehls-Schnellpfad → LLM-Inferenz → Nachfrage zur Klärung.
- Natürlichsprachliche Interaktion mit Abonnenten (keine Befehlssyntax zu erlernen).
- Teilnehmerzuordnungsvertrag, der `created_by/assigned_to` + `@`-Tagging steuert.
- Echter Docs Chain-Konsument (66-Dokumenten-Korpus, Mehrformat-Unterstützung, verifiziert).

Herausforderungen & Lösungen

- **Mehrdeutige natürlichsprachliche Intents:** Gelöst durch die dreistufige Erkennungs-/Inferenz-/Klarungsleiter statt blindem Raten.
- **Zuverlässige Verteilung:** Gelöst durch ein Ingestions-→Multi-Channel-Dispatch-Design, sodass Alarme ihr Ziel erreichen.
- **Korrekte Zuordnung über Kanäle hinweg:** Gelöst durch die Umgebungsvariable für Bediener-Benutzernamen und den Teilnehmerzuordnungsvertrag.
- **Dokumentationsabweichung:** Gelöst durch die Anbindung des Dokumentationskorpus an Docs Chain mit verifizierten Transformationen.

Technologie-Stack (Warum + Wie)

- **Go** – Kernlogik für Events und Dispatching (an organisationsspezifische Sprachmuster angepasst).
- **Shell** – Bediener-Tools und Setup-Skripte.
- **Claude-Code (LLM)** – Inferenzschicht für Freitextnachrichten.
- **Messenger-Channel-Adapter** – Multi-Channel-Benachrichtigungsverteilung.
- **Docs Chain** – Dokumentations-Build-/Verifizierungspipeline (Markdown→HTML/PDF/DOCX).
- **Helix-Constitution-Submodul** – Geerbte Governance/Regeln.